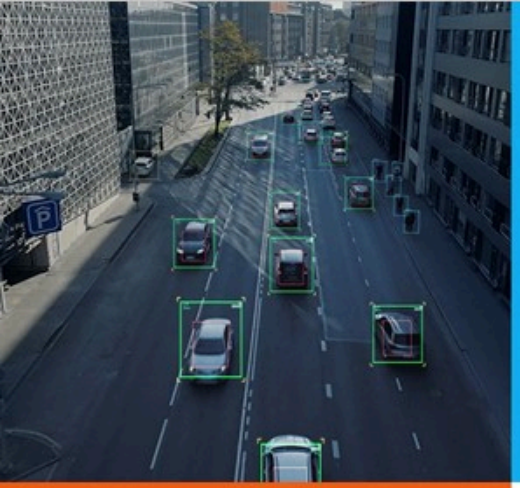


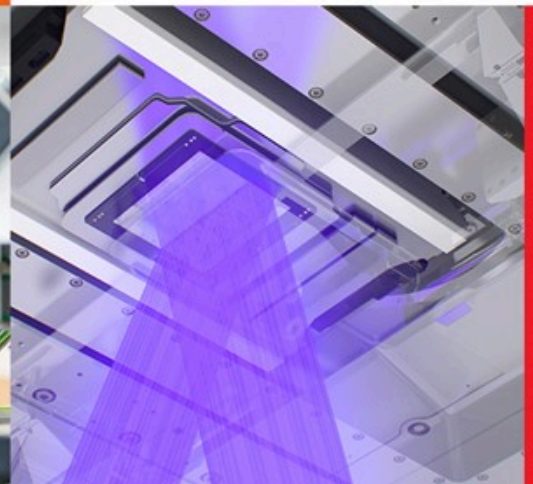
MECHATRONICS

MATHWARE

ASSEMBLY

SIOUX TECHNOLOGIES

ELECTRONICS



SOFTWARE

Ranges

Ranges

- problems with iterators
- range
- views and adaptors
- sentinels
- P3411R0

© Sioux Technologies | Confidential

Fundamentally unsafe

There are no built-in safety checks or sentinels to prevent their use after invalidation

```
std::vector<int> v{1, 2, 3};
auto it = v.begin();
v.push_back(4);   // may reallocate, invalidates iterators
int x = *it;      // undefined behaviour
```

© Sioux Technologies | Confidential

Sentinels

Before ranges, iterators always came in pairs of the same type — begin and end.

This makes some patterns awkward (e.g., searching until a terminator).

```
// Can't stop at '\0' without computing std::end(...)
for (auto it = std::begin(arr); it != std::end(arr); ++it) {
    ...
}
```

© Sioux Technologies | Confidential

Composability

Iterators don’t compose well.

You typically end up writing loops instead of combining operations.

```
std::vector<int> result;
for (auto it = v.begin(); it != v.end(); ++it) {
    if (!predicate(*it)) continue;
    auto transformed = transform(*it);
    if (stop_condition(transformed)) break;
    result.push_back(transformed);
}
```

© Sioux Technologies | Confidential

Errors

If find fails, you must manually compare with end().

Forgetting this = bugs.

Example

```
auto it = std::find(v.begin(), v.end(), 99);
int x = *it;   // BUG if element not found
```

© Sioux Technologies | Confidential

Iterator Invalidation

Different containers have different invalidation rules.

Container	push_back invalidates?
`std::vector`	Yes (on reallocation)
`std::list`	Never

© Sioux Technologies | Confidential

ex1.cpp:

Convert a raw for loop to an STL algorithm

rediscover the pain of using iterators.

© Sioux Technologies | Confidential

What is a range

a range is any object that can produce a begin and an end.

```
template< class T >
concept range = requires( T& t ) {
    ranges::begin(t);
    ranges::end (t);
};
```

© Sioux Technologies | Confidential

- Make iteration explicit and unified
- Improve safety and clarity
- Reduce manual iterator handling
- Allow algorithms to work directly on containers

```
std::sort(v.begin(), v.end());
std::sort(v);
```

© Sioux Technologies | Confidential

```
template< ranges::random_access_range R, class Comp = ranges::less,
          class Proj = std::identity >
requires std::sortable<ranges::iterator_t<R>, Comp, Proj>
constexpr ranges::borrowed_iterator_t<R>
    sort( R&& r, Comp comp = {}, Proj proj = {} );
```

© Sioux Technologies | Confidential

Projection

Remove boilerplate using projection

```
struct Lad {
    std::string name;
    int age;
};
int main() {
    std::vector<Lad> theLads = { {"Erik", 77}, {"Bob", 33}, {"Charlie", 53} };
    std::ranges::sort(theLads,
        [](const auto& a, const auto& b) { return a.age > b.age; });
    std::ranges::sort(theLads, std::ranges::greater{}, &Lad::age);
}-
```

© Sioux Technologies | Confidential

- pass through the projection first then the comparator.
- Invokable: Projections can be member pointers, function pointers, or lambdas.
- Performance: Projections are called twice per comparison (once for each side).

© Sioux Technologies | Confidential

sentinel

```
bool operator==(const char* it, semicolon_sentinel)
{
    return *it == ';';
}

const char* text = "abc;def";
auto r = std::ranges::subrange(
    text,                   // iterator
    semicolon_sentinel{}    // sentinel
);
```

© Sioux Technologies | Confidential

views

A View is a range that is

- Cheap to copy
- Probably does not own its elements

```
template<class T>
concept view = ranges::range<T> && std::movable<T> && ranges::enable_view<T>;

auto get_first_three_view(std::vector<int>& v) {
    return std::ranges::subrange(v.begin(), v.begin() + 3);   // view
}
```

© Sioux Technologies | Confidential

The concept of views was invented for composability:

- range → transform → filter → slice → consume
- Do not want to copy underlying data when passing into the next algorithm

© Sioux Technologies | Confidential

```
std::vector<int> v{1, 2, 3, 4, 5, 6};
auto r1 = std::ranges::subrange(v);
auto r2 = std::ranges::take_view(r1, 4);
auto r3 = std::ranges::filter_view(r2,
    [](int x){ return x % 2 == 0; });
for (int x : r3)
    std::cout << x << " ";
```

© Sioux Technologies | Confidential

Pipeline syntax

temporary views are created behind the screen

```
std::vector<int> v{1, 2, 3, 4, 5, 6};

//type of r is a view
auto r = v
        | std::ranges::views::take(4)
        | std::ranges::views::filter([](int x){ return x % 2 == 0; });

for (int x : r)
    std::cout << x << " ";
```

© Sioux Technologies | Confidential

Adaptors

A factory object

Examples:

- std::views::filter
- std::views::transform
- std::views::take

© Sioux Technologies | Confidential

What is the real type?

this compiles:

```
#include <ranges>
#include <vector>
#include <iostream>

int main()
{
    std::vector<int> v = {1,2,3,4};

    auto is_even = [](int x){ return x % 2 == 0; };
    std::ranges::filter_view fv(v, is_even);

    for (int x : fv) {
        std::cout << x << "\n";
    }
}
```

© Sioux Technologies | Confidential

Only possible because of Class Template Argument Deduction (CTAD, C++17) .

Not possible as return value. rv must be fully qualified (or deduced).

```
template< ranges::input_range V,
          std::indirect_unary_predicate<ranges::iterator_t<V>> Pred >
    requires ranges::view<V> && std::is_object_v<Pred>
class filter_view
    : public ranges::view_interface<filter_view<V, Pred>>
```

© Sioux Technologies | Confidential

The range is wrapped in a ref_view by the adaptor to ensure a copy is lightweight.

Can fully qualify.

```
std::vector<int> v = {1,2,3,4};
auto is_even = [](int x){ return x % 2 == 0; };
auto factory = std::views::filter(is_even);
auto view = v | factory;
static_assert(
    std::same_as<
        decltype(view),
        std::ranges::filter_view<std::ranges::ref_view<decltype(v)>, decltype(is_even)>
    >
);
```

© Sioux Technologies | Confidential

Views try to be cheap to copy but dont have to be

```
int main()
{
    std::vector<int> v = {1, 2, 3};
    // l-value case
    auto view1 = std::views::all(v);
    static_assert(
        std::is_same_v<decltype(view1), std::ranges::ref_view<std::vector<int>>>,
        "Expected ref_view for lvalue"
    );
    // r-value case
    auto view2 = std::views::all(std::vector<int>{1, 2, 3});
    static_assert(
        std::is_same_v<decltype(view2), std::ranges::owning_view<std::vector<int>>>,
        "Expected owning_view for rvalue"
    );
    return 0;
}
```

© Sioux Technologies | Confidential

rewrite ex1 using ranges.

ex2.cpp

© Sioux Technologies | Confidential

Custom views and adaptors

Create a moving average algorithm. given a range of numbers A. produce a new range B with the moving average of the numbers in A.

need some kind of stateful iterator, not provided by std

```
int main() {
    std::vector<double> data = {1, 2, 3, 4, 5, 6};
    auto avg3 = data | moving_average(3);
    return 0;
}
```

© Sioux Technologies | Confidential

concept used to constrain templated parameter r

```
struct moving_average {
    std::size_t window_;
    friend auto operator|(std::ranges::viewable_range auto&& r, moving_average const& self) {
        return moving_average_view(std::forward<decltype(r)>(r), self.window_);
    }
};
```

© Sioux Technologies | Confidential

```
template <std::ranges::input_range V>
requires std::ranges::view<V> && std::is_arithmetic_v<std::ranges::range_value_t<V>>
class moving_average_view
    : public std::ranges::view_interface<moving_average_view<V>>
{
    V base_;
    std::size_t window_;
    using T = std::ranges::range_value_t<V>;
public:
    template <std::ranges::viewable_range R>
    moving_average_view(R&& r, std::size_t w)
        : base_(std::forward<R>(r)), window_(w) {}
    struct iterator {
        using value_type = T;
        using difference_type = std::ptrdiff_t;
        using iterator_category = std::input_iterator_tag;
        std::deque<T> buf_;
        iterator() = default;
        iterator(std::ranges::iterator_t<V> it,
                 std::ranges::sentinel_t<V> end,
                 std::size_t window)
            : it_(it), end_(end), window_(window) {
        }
        value_type operator*() const {
        }
        iterator& operator++() {
        }
    };
    iterator begin() {
        return iterator(std::ranges::begin(base_), std::ranges::end(base_), window_);
    }
    std::default_sentinel_t end() const noexcept {
        return {};
    }
};
```

© Sioux Technologies | Confidential

The power of std::views::all_t

CTAD rule, deduce parameter V from parameter R.

```
template <class R>
moving_average_view(R&&, std::size_t)
-> moving_average_view<std::views::all_t<R>>;
```

std::views::all_t deduces to ref_view or owning_view depending on R.

```
int main() {
    std::vector<double> data = {1, 2, 3, 4, 5, 6};
    auto avg3 = data | moving_average(3);
    static_assert(std::same_as<decltype(avg3), moving_average_view<std::ranges::ref_view<std::vector<double>>>>);
    auto avg4 = std::move(data) | moving_average(3);
    static_assert(std::same_as<decltype(avg4), moving_average_view<std::ranges::owning_view<std::vector<double>>>>);
    return 0;
}
```

© Sioux Technologies | Confidential

Adapter closure now only works in pipeline

Free pipeline operator --> inherit from std::ranges::range_adaptor_closure

```
struct moving_average : std::ranges::range_adaptor_closure<moving_average>
{
    std::size_t window_;
    auto operator()(std::ranges::viewable_range auto&& r) const {
        return moving_average_view(std::forward<decltype(r)>(r), window_);
    }
};
int main() {
    std::vector<double> data = {1,2,3,4,5,6};
    auto avg_pipe = data | moving_average{.window_ = 3};
    static_assert(std::same_as<decltype(avg_pipe), moving_average_view<std::ranges::ref_view<std::vector<double>>>>);
    moving_average ma{.window_ = 3};       // closure object
    auto avg3 = ma(data);                  // call operator() directly
    static_assert(std::same_as<decltype(avg3), moving_average_view<std::ranges::ref_view<std::vector<double>>>>);
}
```

© Sioux Technologies | Confidential

ex3.cpp

© Sioux Technologies | Confidential

C++ Ranges have a "Type-Safe Tombstone"

```
auto get_data() { return std::vector{1, 2, 3}; }
int main()
{
    // std::ranges::find returns std::ranges::dangling.
    auto it = std::ranges::find(get_data(), 2);
    return *it;
}
```

© Sioux Technologies | Confidential

Runtime UB error intercepted by the compiler:

```
<source>: In function 'int main()':
<source>:9:12: error: no match for 'operator*' (operand type is 'std::ranges::dangling')
    9 |     return *it;
      |            ^~~
Compiler returned: 1
```

© Sioux Technologies | Confidential

Why does find return dangling while views::all returns an owning_view?

- Algorithms return iterators. They cannot own the container.
- Views are objects. When given an Rvalue, they move the data into an owning_view.

```
//moving_average_view<std::ranges::owning_view<std::vector<int>>>
auto v = get_data() | std::views::filter(is_even);
// 'it' cannot hold the vector. To prevent a crash, it becomes 'dangling'.
auto it = std::ranges::find(get_data(), 2);
```

© Sioux Technologies | Confidential

Observation: Not all r-value ranges are unsafe

If the range dies, the iterators remain valid

```
auto it = std::ranges::find(std::string_view{"Hello"}, 'e');
static_assert(!std::same_as<decltype(it), std::ranges::dangling>);
```

© Sioux Technologies | Confidential

```
template<typename T>
struct MyCursor {
    T* ptr;
    T* begin() const { return ptr; }
    T* end() const { return ptr + 10; }
};
template<typename T>
inline constexpr bool std::ranges::enable_borrowed_range<MyCursor<T>> = true;
int main() {
    auto it = std::ranges::find(MyCursor<int>{nullptr}, 42);
    return *it;
}
```

© Sioux Technologies | Confidential

```
template<std::ranges::input_range R>
decltype(auto) get_first(R&& r) {
    if constexpr (std::ranges::borrowed_range<R>) {
        return *std::ranges::begin(r); // Returns Reference (Fast)
    } else {
        using V = std::ranges::range_value_t<R>;
        return static_cast<V>(*std::ranges::begin(r)); // Returns Copy (Safe)
    }
}
int main() {
    decltype(auto) val = get_first(std::vector{1, 2});
}
```

© Sioux Technologies | Confidential

Custom sentinel: keeps iterating until std::optional is empty

```
struct element_iterator {
    explicit element_iterator(const char* p)
        : cur_(p) {
        current_ = read_next();
    }
    const Element& operator*() const { return *current_; }
    element_iterator& operator++() {
        current_ = read_next();
        return *this;
    }
    friend bool operator==(const element_iterator& it, std::default_sentinel_t) {
        return !it.current_.has_value();
    }
private:
    const char* cur_ = nullptr;
    std::optional<Element> current_;
    std::optional<Element> read_next() { }
};
```

© Sioux Technologies | Confidential

std::ranges::view_interface

Avoiding boilerplate

.empty(), .size(), .front(), .back

```
class element_range : public std::ranges::view_interface<element_range> {
public:
    explicit element_range(const char* text) : text_(text) {}
    element_iterator begin() const { return element_iterator{text_}; }
    std::default_sentinel_t end() const { return {}; }
private:
    const char* text_;
};
```

© Sioux Technologies | Confidential

Compile-time

- Heavy lifting is done by the type system.
- building the pipeline is done at compile time
- runtime; iterating over the items.
- You could get many specializations for a view (moving_average_view)
- code size can explode
- compile times can explode, any_view: P3411R0

© Sioux Technologies | Confidential

ex4.cpp

ex5.cpp

ex6.cpp

© Sioux Technologies | Confidential

We bring **high-tech** to life